

TỔNG HỢP LÝ THUYẾT & GIẢI THUẬT ĐỒ THỊ

Tài liệu học tập tổng hợp từ Playlist Bài giảng Lý thuyết Đồ thị (28tech)

1. BÀI TOÁN DUYỆT ĐỒ THỊ CƠ BẢN (DFS & BFS)

1.1. Thuật toán Tìm kiếm theo chiều sâu (DFS - Depth-First Search)

- Nguyên lý:** DFS ưu tiên đi sâu nhất có thể theo từng nhánh của đồ thị trước khi quay lại (backtrack). Thuật toán được cài đặt tự nhiên bằng **Đệ quy (Recursion)** hoặc sử dụng cấu trúc dữ liệu **Ngăn xếp (Stack)**.
- Trạng thái:** Sử dụng mảng visited[] để đánh dấu các đỉnh đã thăm, tránh lặp vô hạn.
- Độ phức tạp:** Thời gian $O(V + E)$ đối với danh sách kề, $O(V^2)$ đối với ma trận kề.
- Ứng dụng trong Playlist:** Kiểm tra tính liên thông, tìm đường đi, phát hiện chu trình, kiểm tra đỉnh trụ và cạnh cầu.

1.2. Thuật toán Tìm kiếm theo chiều rộng (BFS - Breadth-First Search)

- Nguyên lý:** BFS duyệt đồ thị theo từng 'lớp' (level) khoảng cách từ đỉnh xuất phát. Thuật toán sử dụng cấu trúc dữ liệu **Hàng đợi (Queue - FIFO)**.
- Tính chất đặc trưng:** BFS luôn tìm ra **đường đi ngắn nhất** (theo số lượng cạnh) từ đỉnh nguồn tới mọi đỉnh khác trên đồ thị không có trọng số.
- Độ phức tạp:** Thời gian $O(V + E)$ trên danh sách kề.
- Ứng dụng trong Playlist:** Tìm đường đi ngắn nhất không trọng số, kiểm tra đồ thị 2 phía (bipartite coloring), thuật toán Kahn cho sắp xếp Topo.

2. TÍNH LIÊN THÔNG VÀ ỨNG DỤNG TRÊN LƯỚI Ô VUÔNG

2.1. Đếm số thành phần liên thông (Connected Components)

- Trên đồ thị vô hướng, một thành phần liên thông là tập hợp các đỉnh cực đại mà giữa 2 đỉnh bất kỳ đều có đường đi.
- Giải thuật:** Duyệt qua tất cả các đỉnh $1 \dots V$. Nếu đỉnh u chưa được thăm, tăng số TPLT lên 1 và gọi DFS(u) / BFS(u) để đánh dấu toàn bộ các đỉnh thuộc cùng TPLT với u .

2.2. Áp dụng DFS / BFS trên lưới ô vuông (Grid Graph / Matrix)

- Lưới ô vuông $M \times N$ được coi là đồ thị với mỗi ô (r, c) là một đỉnh. Hai ô kề nhau có cạnh nối nếu thỏa mãn điều kiện di chuyển (thường là 4 hướng: Trên, Dưới, Trái, Phải hoặc 8 hướng).
- Mảng hướng di chuyển:**
- 4 hướng: $dx[] = \{-1, 1, 0, 0\}$, $dy[] = \{0, 0, -1, 1\}$.
- Tìm đường đi ngắn nhất trong ma trận:** Dùng BFS kết hợp mảng khoảng cách $dist[r][c]$. BFS đảm bảo ô đầu tiên tới đích chính là đường đi ngắn nhất.

3. SẮP XẾP TOPO VÀ KIỂM TRA CHU TRÌNH (CYCLE DETECTION)

3.1. Sắp xếp Topo (Topological Sort - Thuật toán Kahn)

- Định nghĩa:** Sắp xếp thứ tự các đỉnh của đồ thị có hướng sao cho với mỗi cạnh có hướng $u \rightarrow v$, đỉnh u luôn đứng trước đỉnh v . Điều kiện bắt buộc: Đồ thị phải là **DAG (Directed Acyclic Graph)**.
- Thuật toán Kahn (dùng BFS):**
 - Tính bán bậc vào (indegree) của tất cả các đỉnh.

- Đưa tất cả các đỉnh có $\text{indegree} == 0$ vào Hàng đợi (Queue).
- Lấy đỉnh u ra khỏi Queue, thêm vào kết quả Topo. Giảm indegree của các đỉnh kề v . Nếu $\text{indegree}[v] == 0$, đưa v vào Queue.
- Nếu số đỉnh trong kết quả Topo $< V$, đồ thị có chu trình.

3.2. Kiểm tra chu trình trên Đồ thị (Cycle Detection)

- Đồ thị vô hướng:** Dùng DFS/BFS. Trong quá trình duyệt từ u sang v , nếu gặp v đã được thăm và $v \neq \text{parent}[u]$ thì kết luận đồ thị có chu trình.
- Đồ thị có hướng:** Dùng mảng màu/trạng thái (0: Chưa thăm, 1: Đang thăm - trong nhánh đệ quy, 2: Đã hoàn thành). Nếu trong quá trình DFS gặp đỉnh có trạng thái là 1 thì tồn tại chu trình ngược (Back Edge).

4. CÂY KHUNG NHỎ NHẤT (MINIMUM SPANNING TREE - MST)

4.1. Thuật toán Kruskal (Dùng cấu trúc DSU)

- Ý tưởng:** Lần lượt chọn các cạnh có trọng số nhỏ nhất mà không tạo thành chu trình.
- Cấu trúc DSU (Disjoint Set Union):** Sử dụng hai thao tác $\text{find}(u)$ (tìm đại diện tập hợp có tối ưu nén vết) và $\text{union}(u, v)$ (gộp 2 tập hợp theo hợp theo hạng/kích thước).
- Các bước thực hiện:**
 - Sắp xếp danh sách E cạnh theo trọng số tăng dần.
 - Khởi tạo DSU với V tập hợp rời rạc.
 - Duyệt từng cạnh (u, v, w) : Nếu $\text{find}(u) \neq \text{find}(v)$, thêm cạnh vào MST, gộp $\text{union}(u, v)$ và cộng w vào tổng trọng số.
 - Dừng khi đã chọn đủ $V-1$ cạnh. Độ phức tạp: $O(E \log E)$.

4.2. Thuật toán Prim

- Ý tưởng:** Phát triển cây khung từ 1 đỉnh xuất phát tùy ý bằng cách luôn chọn cạnh có trọng số nhỏ nhất nối một đỉnh thuộc cây khung với một đỉnh ngoài cây khung.
- Cài đặt tối ưu:** Sử dụng **Hàng đợi ưu tiên (Priority Queue / Min-Heap)** để lưu trữ các cạnh nối với các đỉnh chưa nằm trong cây khung.
- Độ phức tạp: $O(E \log V)$, rất thích hợp cho đồ thị dày (mật độ cạnh cao).

5. ĐỒ THỊ HAI PHÍA, ĐỈNH TRỤ VÀ CẠNH CẦU

5.1. Thuật toán Kiểm tra Đồ thị Hai phía (Bipartite Graph)

- Định nghĩa:** Đồ thị hai phía là đồ thị mà tập đỉnh có thể phân chia thành 2 tập rời rạc V_1, V_2 sao cho mọi cạnh chỉ nối 1 đỉnh thuộc V_1 với 1 đỉnh thuộc V_2 .
- Tính chất:** Đồ thị là hai phía khi và chỉ khi **không chứa chu trình có độ dài lẻ**.
- Thuật toán tô màu (2-Coloring):** Dùng BFS/DFS tô màu đồ thị bằng 2 màu (1 và 2). Nếu đỉnh u có màu c , tất cả đỉnh kề với u phải có màu $3-c$. Nếu gặp đỉnh kề đã tô màu giống u thì không phải đồ thị hai phía.

5.2. Đỉnh trụ (Khớp) và Cạnh cầu (Bridge & Articulation Point)

- Đỉnh trụ (Khớp):** Đỉnh mà nếu xóa đỉnh đó (cùng các cạnh liên kết với nó) sẽ làm **tăng số thành phần liên thông** của đồ thị.
- Cạnh cầu:** Cạnh mà nếu xóa nó sẽ làm tăng số thành phần liên thông của đồ thị.
- Phương pháp kiểm tra cơ bản trong playlist:** Thử xóa từng đỉnh / từng cạnh và gọi DFS/BFS đếm số TPLT mới để so sánh với số TPLT ban đầu.

6. THUẬT TOÁN ĐƯỜNG ĐI NGẮN NHẤT TRÊN ĐỒ THỊ TRỌNG SỐ

6.1. Thuật toán Dijkstra

- **Phạm vi áp dụng:** Tìm đường đi ngắn nhất từ 1 đỉnh nguồn đến tất cả các đỉnh còn lại trên đồ thị có trọng số **KHÔNG ÂM**.
- **Nguyên lý Greedy:** Luôn chọn đỉnh u có khoảng cách tạm thời $\text{dist}[u]$ nhỏ nhất chưa được cố định, sau đó tiến hành 'thư giãn' (Relaxation) các cạnh đi ra từ u :
if ($\text{dist}[v] > \text{dist}[u] + w$) $\text{dist}[v] = \text{dist}[u] + w$;
- **Cài đặt tối ưu:** Dùng Priority Queue (Min-Heap). Độ phức tạp: $O((V + E) \log V)$.

6.2. Thuật toán Bellman-Ford

- **Phạm vi áp dụng:** Tìm đường đi ngắn nhất trên đồ thị có thể chứa **trọng số ÂM** và kiểm tra **chu trình âm (Negative Cycle)**.
- **Nguyên lý:** Thực hiện thao tác Relaxation trên toàn bộ E cạnh trong $V-1$ lần lặp.
- **Phát hiện chu trình âm:** Nếu ở lần lặp thứ V vẫn còn cạnh có thể thư giãn (giảm $\text{dist}[v]$), đồ thị chứa chu trình âm.
- Độ phức tạp: $O(V \cdot E)$.

7. CHU TRÌNH EULER VÀ ĐƯỜNG ĐI EULER

- **Đường đi Euler:** Đường đi qua **mỗi cạnh** của đồ thị đúng 1 lần.
- **Chu trình Euler:** Đường đi Euler khép kín (đỉnh xuất phát trùng đỉnh kết thúc).
- **Điều kiện tồn tại (Đồ thị vô hướng liên thông):**
 - Chu trình Euler: **TẤT CẢ** các đỉnh đều có **bậc chẵn**.
 - Đường đi Euler: Có đúng **0 hoặc 2 đỉnh bậc lẻ** (nếu có 2 đỉnh bậc lẻ, đường đi bắt đầu từ 1 đỉnh lẻ và kết thúc ở đỉnh lẻ còn lại).
- **Điều kiện tồn tại (Đồ thị có hướng liên thông yếu):**
 - Chu trình Euler: Bán bậc vào bằng bán bậc ra đối với **MỌI** đỉnh ($\text{indegree}[u] == \text{outdegree}[u]$).
- **Thuật toán Hierholzer:** Sử dụng DFS kết hợp Stack để tìm chu trình Euler trong thời gian $O(V + E)$.

8. BẢNG TỔNG HỢP PLAYLIST BÀI GIẢNG (Nguồn: 28tech)

STT Video	Tên Bài Giảng Trong Playlist	Thuật Toán / Chủ Đề Chính	Độ Phức Tạp
#3	Thuật Toán DFS C++	Tìm kiếm theo chiều sâu (DFS)	$O(V + E)$
#4	Giải Thuật BFS	Tìm kiếm theo chiều rộng (BFS)	$O(V + E)$
#5	Đồ Thị Vô Hướng Liên Thông	Đếm số Thành phần liên thông	$O(V + E)$
#6	Tìm Đường Đi Không Trọng Số	Đường đi ngắn nhất không trọng số (BFS)	$O(V + E)$
#7	DFS, BFS Trên Lưới Ô Vuông	Duyệt Ma trận / Lưới ô vuông	$O(M * N)$
#8	Đường Đi Ngắn Nhất Trên Ma Trận	BFS trên ma trận ô vuông	$O(M * N)$
#9	Topological Sort (Kahn's Algo)	Sắp xếp Topo dùng BFS	$O(V + E)$
#10	Kiểm Tra Chu Trình Đồ Thị	Phát hiện chu trình (DFS/BFS/Màu)	$O(V + E)$
#16	Thuật Toán Kruskal & DSU	Cây khung cực tiểu (MST) - Kruskal	$O(E \log E)$
#17, #18	Thuật Toán Prim Tìm MST	Cây khung cực tiểu - Prim & Priority Queue	$O(E \log V)$

STT Video	Tên Bài Giảng Trong Playlist	Thuật Toán / Chủ Đề Chính	Độ Phức Tạp
#19	Kiểm Tra Đồ Thị 2 Phía	Đồ thị lưỡng phân (Bipartite Graph / 2-Coloring)	$O(V + E)$
#20	Đỉnh Trụ và Cạnh Cầu	Khớp và Cầu trong đồ thị	$O(V^*(V+E))$ / Tarjan
#21	Thuật Toán Dijkstra	Đường đi ngắn nhất trọng số không âm	$O((V+E) \log V)$
#22	Chu Trình & Đường Đi Euler	Eulerian Circuit & Path (Hierholzer)	$O(V + E)$
#23	Thuật Toán Bellman-Ford	Đường đi ngắn nhất trọng số âm, Chu trình âm	$O(V * E)$

Ghi Chú Học Tập & Ôn Luyện

- Tất cả các thuật toán trên đều dựa trên nguyên lý biểu diễn đồ thị bằng Danh sách kề (Adjacency List) để tối ưu thời gian và bộ nhớ.
- Khi làm bài tập lập trình thi đấu hoặc phỏng vấn, cần lưu ý đặc điểm của từng thuật toán (ví dụ: Dijkstra không chạy đúng trên trọng số âm, BFS dùng cho đồ thị không trọng số, Kahn yêu cầu DAG).